

Sprawozdanie z projektu zaliczeniowego przedmiotu Systemy Operacyjne Czasu Rzeczywistego

Symulator operacji windy budynkowej w języku Java

Autorzy:

Jan Machowski, 325492
Bartłomiej Modzolewski, 331719
Karolina Wiśniewska, 331757

Semestr: 26L
Język: Java 17+
Środowisko GUI Swing

0 Spis treści

1	Wstęp	2
2	Struktura projektu	3
3	Opis klas	3
3.1	Klasa <code>Entity</code>	3
3.1.1	Opis klasy	3
3.1.2	Opis stanów <code>Entity.State</code>	3
3.1.3	Najważniejsze metody:	4
3.2	Klasa <code>Floor</code>	4
3.2.1	Opis klasy	4
3.2.2	Najważniejsze metody:	4
3.3	Klasa <code>Elevator</code>	5
3.3.1	Opis klasy	5
3.3.2	Opis stanów <code>Elevator.State</code>	5
3.3.3	Kolejka <code>destinationQueue</code>	6
3.3.4	Najważniejsze metody:	6
3.4	Klasa <code>Division</code>	7
3.4.1	Opis klasy	7
3.4.2	Najważniejsze metody:	8
3.5	Klasa <code>ElevatorGUI</code>	8
3.5.1	Wykorzystane układy interfejsu <code>BorderLayout</code>	8
3.5.2	Najważniejsze metody:	9
4	Uruchomienie projektu	10
4.1	Wymagania	10
4.2	Główne repozytorium kodu	10
4.3	Kompilacja i uruchomienie z wiersza poleceń	10
4.4	Uruchomienie z IDE	11
4.5	Konfiguracja parametrów w GUI	11
5	Architektura i przepływ danych	12
6	Napotkane problemy i błędy	12
6.1	Wstępna obsługa kolejki priorytetowej	12
6.2	Omijanie pięter, na których winda mogła się zatrzymać	13
6.3	Długi czas oczekiwania w specyficznych przypadkach	13
7	Dalsze możliwości rozwoju	14
8	Podsumowanie	15

1 Wstęp

Niniejszy dokument stanowi sprawozdanie z projektu zaliczeniowego w ramach zajęć projektowych przedmiotu Systemy Operacyjne Czasu Rzeczywistego, odbywających się w semestrze letnim roku 2026. Projekt polegał na stworzeniu aplikacji symulującej działanie przykładowego, wybranego przez zespół, systemu operacyjnego czasu rzeczywistego, oraz przedstawienie jego działania.

Wybrany przez zespół temat aplikacji to system obsługujący działanie windy budynku. Projekt zakładał stworzenie oprogramowania obsługującego, oraz środowiska symulującego działanie windy, wraz z graficznym interfejsem użytkownika.

Kluczowymi aspektami do rozwiązania w celu ukończenia projektu były:

1. Zapewnienie ciągłego działania systemu windy, niezależnie od liczby oczekujących osób;
2. Wykorzystanie odpowiedniego algorytmu kolejkowania, w celu realistycznego odtworzenia działania systemu;
3. Umożliwienie modyfikacji podstawowych parametrów windy, takich jak:
 - Liczba piętr, piętro początkowe i końcowe;
 - Prędkość przejazdu windy między piętrami;
 - Czas oczekiwania na obsługę wejścia / wyjścia pasażerów;
 - Maksymalna liczba osobników w windzie;
 - Piętro początkowe windy.
4. Stworzenie odpowiedniego środowiska symulacyjnego dla codziennego działania systemu, w co wchodzi m.in. dodawanie pasażerów na obsługiwanych piętrach.
5. Wykonanie funkcjonalnego i intuicyjnego interfejsu graficznego użytkownika w celu obsługi symulacji.

Najistotniejszym elementem całego projektu jest stworzenie wykorzystanie algorytmu kolejkowania. Jak można się domyśleć, winda nie może przykładowo spontanicznie zatrzymać się na piętrze, z którego natychmiast wcześniej zostało zarejestrowane wezwanie, oraz nie powinna ignorować wezwania które zostało wykonane znaczny czas wcześniej.

Kolejnym istotnym elementem było zapewnienie funkcjonalności symulacyjnych windy - zarówno manualnych, jak i automatycznych. W to wchodzi m.in. ręczne oraz losowe tworzenie pasażerów czekających na odpowiednich piętrach, obsługa ich zapytań tak, żeby jak najwięcej osób dostało się na swoje wymagane piętra w odpowiednim czasie a także unikanie przypadku, w którym winda kulkukrotnie przejeżdża przez piętro bez zatrzymania się na nim.

Wynikiem projektu jest załączony w katalogu `src/` projekt **Elevator**, który jest skończoną implementacją symulacji systemu operacyjnego czasu rzeczywistego. Ni-

niejsze sprawozdanie opisywać będzie strukturę, działanie, wykorzystanie oraz kluczowe aspekty napotkane podczas tworzenia projektu.

2 Struktura projektu

Projekt podzielony jest na dwa pakiety:

- `elevator.model` - logika backend'owa symulacji - model danych, klas wykorzystanych w projekcie i algorytmy sterowania windą i obsługi przypadków;
- GUI - graficzny interfejs użytkownika oparty na bibliotece Swing.

Poniższa tabela przedstawia klasy stworzone w ramach implementacji projektu wraz z ich przynależnością:

Klasa	Pakiet	Rola
Entity	<code>elevator.model</code>	Reprezentacja pasażera
Floor	<code>elevator.model</code>	Piętro budynku
Elevator	<code>elevator.model</code>	Winda
Division	<code>elevator.model</code>	Pion budynku (zbiór n pięter i jednej windy)
ElevatorGUI	GUI	Okno aplikacji Swing

3 Opis klas

3.1 Klasa Entity

3.1.1 Opis klasy

Klasa reprezentująca pojedynczego pasażera (osobnika) w symulacji. Każda instancja posiada 4 istotne informacje:

1. **Unikalny identyfikator**, nadawany automatycznie za pomocą licznika atomowego (`AtomicInteger`);
2. `originFloor` - piętro, z którego pochodzi i na którym oczekuje na windę pasażer. Jest wstawiane w kolejkę `destinationQueue` gdy pasażer się pojawia, oraz usuwane z niej gdy pasażer wsiada do windy;
3. `destinationFloor` - piętro, na którym pasażer chce wysiąść. Jest wstawiane w kolejkę `destinationQueue` gdy pasażer wsiada do windy oraz usuwane gdy pasażer wysiada na wybranej windzie;
4. **Jeden z trzech stanów** `Entity.State`, opisanych poniżej.

3.1.2 Opis stanów Entity.State

Każdy osobnik klasy `Entity` posiada jeden z trzech stanów `Entity.State` opisujących aktualne zadanie pasażera:

- `WAITING_ON_FLOOR` - pasażer oczekuje na windę na piętrze początkowym;

- `IN_ELEVATOR` - pasażer znajduje się w kabinie windy, jedzie na piętro docelowe;
- `ARRIVED` - pasażer dotarł do celu i opuścił windę. Po opuszczeniu windy, obiekt klasy `Entity` jest usuwany.

Obiekty klasy `Entity` zawarte są we wszystkich trzech klasach funkcjonalnych, tj. `Elevator`, `Floor` i `Division`.

3.1.3 Najważniejsze metody:

`boardElevator()`

Zmienia stan pasażera z `WAITING_ON_FLOOR` na `IN_ELEVATOR`. Wyrzuca `IllegalStateException`, gdy pasażer nie oczekuje na piętrze;

`exitElevator()`

Zmienia stan z `IN_ELEVATOR` na `ARRIVED`. Analogicznie zabezpieczona walidacją aktualnego stanu pasażera, co `boardElevator`;

`getId()`, `getOriginFloor()`, `getDestinationFloor()`, `getState()`

Funkcje zwracające wartości pól niezmiennych - identyfikatora, piętra startowego, docelowego oraz aktualnego stanu.

3.2 Klasa `Floor`

3.2.1 Opis klasy

Klasa reprezentująca jedno piętro budynku, do którego dostępna jest winda. Implementuje `Comparable<Floor>` na podstawie numeru piętra, dzięki czemu piętra można porównywać i sortować (używane w `Division` do wyznaczania `minFloor` i `maxFloor`). Klasa ta zawiera:

1. `floorNumber` - numer piętra w kondygnacji;
2. `waitingEntities` - lista pasażerów oczekujących na danym piętrze. Gdy winda zatrzyma się na danym piętrze, oraz nie jest w pełni zapełniona, obiekty osobników z tej listy trafiają do listy `occupants` klasy `Elevator`;

3.2.2 Najważniejsze metody:

`spawnEntity(int destinationFloor)`

Generuje nowy obiekt klasy `Entity` - pasażera na piętrze z podanym piętrem docelowym, dodaje go do listy oczekujących i zwraca referencję. Wyrzuca `IllegalArgumentException` przy próbie stworzenia pasażera z piętrem docelowym równym bieżącemu;

`entityEntersElevator(Entity entity)`

Usuwa pasażera z listy oczekujących i wywołuje `entity.boardElevator()` na danym pasażerze. Wyrzuca `IllegalArgumentException`, gdy pasażer nie należy do tej listy.

`entityExitsElevator(Entity entity)`

Wywołuje `entity.exitElevator()` - pasażer dotarł na miejsce i "znika" z okolicy windy.

`hasWaitingEntities(), getWaitingCount(), getWaitingEntities()`

Metody informacyjne o stanie piętra; `getWaitingEntities()` zwraca niemodyfikowalny widok listy.

3.3 Klasa Elevator

3.3.1 Opis klasy

Klasa `Elevator` jest główną klasą logiki symulacji. Zarządza windy znajdującą się w kondygnacji `Division`: przechowuje informacje o jej aktualnej pozycji, listę pasażerów wewnątrz kabiny, kolejkę pięter docelowych oraz stan ruchu.

Winda porusza się synchronicznie w dedykowanym wątku, blokując go podczas symulacji czasu przejazdu i postoju (`Thread.sleep()`).

Klasa ta zawiera następujące informacje:

1. `destinationQueue` - kolejka priorytetowa klasy `Elevator`. Jest to główna część aplikacji, na której opiera się cała interpretacja systemu operacyjnego czasu rzeczywistego. Do tej kolejki dodawane są piętra, które winda ma odwiedzić, w kolejności ich istotności - opisane w sekcji 3.3.3.
2. `currentState` - aktualny stan windy, wybrany jako jeden spośród dostępnych stanów `Elevator.State` - opisane w sekcji 3.3.2;
3. `currentFloor` - aktualne piętro (numer piętra, nie obiekt klasy `Floor`), na którym znajduje się winda;
4. `speedMs` - prędkość, z jaką winda porusza się między piętrami. W praktyce jest to czas oczekiwania między "przeskoczeniem" windy między piętrami;
5. `IOwaitMs` - czas, jaki winda czeka podczas wchodzenia / wychodzenia pasażerów. Symuluje to m.in. otwarcie drzwi i wymianę pasażerów;
6. `maxCapacity` - maksymalna liczba pasażerów windy dozwolona w konkretnym momencie w czasie;
7. `occupants` - lista klasy `Entity`, będąca reprezentacją aktualnych pasażerów windy.

Interfejs ElevatorListener: Wzorzec obserwatora umożliwiający powiadamianie zewnętrznych komponentów (np. GUI) o każdej zmianie stanu windy. Rejestracja przez `addListener`, wyrejestrowanie przez `removeListener`.

3.3.2 Opis stanów Elevator.State

W celu określenia aktualnego stanu (stateczny / poruszania się), jak i kierunku poruszania się windy, stworzono podklasę `Elevator.State`. Poniżej znajduje się opis stanów zawartych w ramach tej klasy:

- UP - winda porusza się w górę kondygnacji;
- DOWN - winda porusza się w dół kondygnacji;
- IDLE - winda stoi - nie ma pasażerów, ani osób oczekujących na piętrach.

3.3.3 Kolejka `destinationQueue`

Kolejka `destinationQueue` jest najistotniejszym elementem całej implementacji. W praktyce, jest to lista dwuelementowych list postaci

`[numer_piętra, priorytet_piętra]`. Wartość priorytetu jest zależna od tego, w jaki sposób dane piętro zostało dodane do kolejki:

- 1 - wpis dodany na koniec kolejki, wywołanie przez osobnika z poziomu piętra (funkcja `callElevator`);
- -1 - wpis wstawiony ze średnim priorytetem, wywołane przez osobnika w kabinie jako jego piętro docelowe;
- -2 - wpis aktualnie obsługiwany, wstawiony z największym priorytetem.

Największa potencjalna liczba elementów kolejki `destinationQueue` to liczba wszystkich pięter, z czego maksymalnie `maxCapacity` z nich ma ujemną (czyli priorytetową) wartość priorytetu.

3.3.4 Najważniejsze metody:

`processQueue(Division division)`

Pętla główna symulacji. Funkcja przetwarza kolejkę `destinationQueue` do wyczerpania, i jest wywoływana gdy do pustej kolejki zostanie dodane nowe żądanie. W każdej iteracji funkcji wykonywana jest następująca kolejność instrukcji:

1. Głowę kolejki `destinationQueue`, tj. piętro o priorytecie -2 - czyli to, do którego winda musi dojechać zanim zmieni kierunek / zatrzyma jazdę;
2. Wyznaczany jest `currentState` - kierunek ruchu na podstawie pozycji `currentFloor` względem głowy `destinationQueue`;
3. Dopóki `currentFloor` nie równe jest piętru z głowy `destinationQueue`, winda jest przesuwana o jedno piętro (funkcja `moveOneFloor`) i obsługuje postój (`handleFloorStop`), jeśli bieżące piętro jest **gdziekolwiek** na liście docelowej. Tutaj może wydarzyć się jedna z trzech sytuacji:
 - Do windy wchodzi osoba oczekująca na piętrze - aktualizacja `occupants` windy i `waitingEntities` piętra oraz dodanie `destinationFloor` osoby na koniec elementów z priorytetem -1 (czyli między ostatnim elementem -1 a pierwszym elementem +1, jeżeli taki istnieje);
 - Z windy wychodzi osoba - aktualizacja `occupants` windy i usunięcie `destinationFloor` z `destinationQueue`;
 - Winda jest zapełniona - nie zmienia się stan `occupants` ani `destinationQueue`.

4. Gdy kolejka dojdzie do piętra z głowy `destinationQueue`, **co najmniej jeden pasażer na pewno wysiada**, usunięta jest aktualna głowa i ustalona jest nowa głowa kolejki `destinationQueue`.

`enqueueDestination(int floor)`

Dodaje piętro na koniec kolejki, jeśli nie ma go jeszcze w kolejce i jest różne od aktualnego piętra. Wykonywane w ramach funkcji `callElevator`.

`pushDestinationFront(int floor)`

Wstawia piętro z wysokim priorytetem (tj. przed wszystkimi wpisami bez priorytetu ujemnego). Stosowane, gdy pasażer wsiada i jego piętro docelowe znajduje się już w kolejce - wtedy powinno być obsłużone wcześniej, więc jest przesuwane z wyższym priorytetem.

`handleFloorStop(Division division, int floorNum)`

Druga najważniejsza funkcja aplikacji. Obsługuje sytuację windy w czasie postoju na piętrze: najpierw sprawia, że pasażerowie z bieżącym piętrem docelowym wysiadają z windy (usuwa ich z listy `occupants`). Następnie sprawia, że oczekujący na piętrze pasażerowie wsiadają (o ile nie przekroczono pojemności `maxCapacity`), dodaje ich piętra docelowe priorytetowo do kolejki i symuluje czas postoju (`IOwaitMs`).

`isFull(), isEmpty()`

Sprawdzają, czy kabina windy jest zapełniona do maksymalnej pojemności, czy jest pusta.

`getDirection()`

Zwraca aktualny kierunek ruchu na podstawie stanu `Elevator.State` (UP/DOWN/IDLE).

3.4 Klasa `Division`

3.4.1 Opis klasy

Klasa reprezentująca pion budynku. Jako pion rozumie się zbiór (listę) pięter obsługiwanych przez jedną windę. Łączy w sobie obiekty `Floor` z obiektem `Elevator`, zawierając jednocześnie obiekty klasy `Entity` i udostępnia wysokopoziomowe API do wywoływania windy.

Klasa ta zawiera następujące informacje:

1. `floors` - Lista pięter należących do kondygnacji budynku. Na jej podstawie wyznaczane są dostępne dla windy piętra, w tym najwyższe i najniższe dostępne;
2. `elevator` - Obiekt klasy `Elevator`, reprezentujący faktyczną windę działającą windę;
3. `name` - nazwa kondygnacji (atrybut przestarzały, pozostawiony po testach w terminalu);

3.4.2 Najważniejsze metody:

`Division(String name, int startFloor, int endFloor, Elevator elevator)`

Konstruktor, w którym tworzona jest lista pięter od `startFloor` do `endFloor` (włącznie). Wyrzuca `IllegalArgumentException`, gdy `endFloor < startFloor`. Do tego dodawana jest kopia obiektu klasy `Elevator`.

`callElevator(int originFloor, int destinationFloor)`

Główna metoda obsługi windy: tworzy pasażera na piętrze `originFloor` (wywołując `Floor.spawnEntity`) i dodaje `originFloor` do kolejki windy z priorytetem 1 (wywołując `Elevator.enqueueDestination`). Zwraca referencję do nowo utworzonego pasażera.

`getFloor(int floorNumber)`

Metoda zwracająca obiekt `Floor` dla podanego numeru piętra. Wyrzuca `IndexOutOfBoundsException` przy numerze spoza zakresu.

`getMinFloor(), getMaxFloor()`

Wyznaczają i zwracają odpowiednio najniższe i najwyższe piętro w pionie, korzystając z funkcjonalności `Collections.min` i `Collections.max`.

`printStatus()`

Przestarzała metoda Wypisuje na standardowe wyjście aktualny stan wszystkich pięter oraz windy - przestarzałe narzędzie diagnostyczne przed stworzeniem graficznego interfejsu użytkownika.

3.5 Klasa ElevatorGUI

Główne okno interfejsu graficznego aplikacji zbudowane wykorzystując pakiet Swing. W celu utworzenia przejrzystego interfejsu użytkownika, wykorzystano obiekty klasy `JFrame`, które odpowiadają za wizualizację odpowiednich parametrów, zarówno wejściowych jak i wyjściowych aktualnego stanu symulacji w czasie rzeczywistym oraz pozwalają na interakcję z użytkownikiem.

Komunikacja z warstwą modelu odbywa się przez interfejs `ElevatorListener` - obiekt klasy `Elevator` wywołuje `updateUIState()` po każdym ruchu i postoju.

3.5.1 Wykorzystane układy interfejsu BorderLayout

W głównej framudze `JFrame` ustawiono elementy w sposób opisany poniżej:

- `BorderLayout.NORTH` - panel parametrów symulacji. W to wchodzi kontrolki dot. zakresu pięter, prędkości, czas postoju oraz pojemności. W celu zaakceptowania zmian należy wcisnąć przycisk *Zastosuj i restartuj*. **Uwaga** - po wciśnięciu przycisku kolejka symulacji zostanie zresetowana do domyślnego przykładu, usuwając istniejące obiekty `Entity`. (patrz `initSimulationFromInputs` w sekcji 3.5.2)
- `BorderLayout.CENTER` - panel środkowy zawiera dwie wewnętrzne kolumny:

1. listę pięter w kondygnacji Division z aktualną pozycją windy, będącą panelem przedstawiającym rzeczywistą pracę symulacji systemu operacyjnego;
2. listę informacji dot. aktualnego stanu windy i symulacji, w tym: stan symulacji, aktualne piętro na którym jest winda, prędkość przejścia windy między piętrami, stan windy (według 3.3.2), liczba pasażerów w windzie oraz aktualna kolejka z priorytetami;

panel stanu windy (piętro, prędkość, kierunek, pasażerowie, kolejka),

- `BorderLayout.SOUTH` - panel dolny składa się z dwóch sekcji, mianowicie:

1. sekcja ręcznego dodawania pasażerów, z doбором pięter;
2. sekcja kontroli symulacji oraz dodania losowego pasażera.

Rysunek 1: Interfejs graficzny użytkownika, wedle opisu podanego powyżej. W tym przykładzie, winda znajduje się w domyślnej startowej pozycji (piętro 4). Konfiguracja również jest domyślna.

3.5.2 Najważniejsze metody:

`initSimulationFromInputs()`

Metoda inicjalizująca symulację na podstawie podanych parametrów, jednocześnie resetująca kolejkę `destinationQueue` do jej wstępnego stanu. W praktyce, odczytuje wartości z panelu parametrów, tworzy nowy obiekt `Elevator`

i `Division`, rejestruje listener GUI oraz dodaje domyślnych pasażerów startowych;

`startSimulationThread()`

Metoda uruchamiająca wątek symulacji, jeśli nie jest aktywny. Wątek ten wywołuje `Elevator.processQueue`, czyli kontunuuje główną pętlę programu;

`stopSimulation()`

Ustawia flagę `stopRequested` i przerywa wątek symulacji wykorzystując do tego `Thread.interrupt()`;

`spawnRandomPassenger()`

Metoda automatycznego generowania i wstawiania osobnika do kondygnacji. Losuje parę różnych pięter w ramach aktualnej listy pięter w obiekcie klasy `Division` i wywołuje `callElevator` dla nowego `Entity` z danymi piętrami. Jeśli winda stoi i symulacja jest aktywna, automatycznie wznawia wątek;

`addManualPassenger()`

Metoda pobrania wartości ze spinnerów *Z piętra* i *Na piętro* i wywołuje `callElevator`. Waliduje, czy piętra są różne;

`updateUIState()`

Metoda aktualizuje wszystkich etykiet i elementów zmiennych w graficznym interfejsie użytkownika. Aktualizuje listę pięter za pomocą `SwingUtilities.invokeLater`, zapewniając wywołanie na wątku EDT.

4 Uruchomienie projektu

4.1 Wymagania

- **Java Development Kit (JDK)** w wersji 17 lub nowszej.
- System operacyjny z obsługą Java Swing (Windows, Linux z X11/Wayland, macOS).

4.2 Główne repozytorium kodu

Cały projekt dostępny jest publicznie w repozytorium https://github.com/crppl/rtos_projekt. W celu wykonania projektu należy sklonować repozytorium za pomocą komendy:

```
1 git clone https://github.com/crppl/rtos_projekt
```

Listing 1: Sklonowanie repozytorium

4.3 Kompilacja i uruchomienie z wiersza poleceń

Po prawidłowym skopiowaniu repozytorium, w celu uruchomienia projektu z wiersza poleceń należy wejść do głównego katalogu repozytorium oraz wykonać następujące komendy:

```

1 cd src/main/java
2 javac -d out elevator/model/*.java GUI/ElevatorGUI.java
3 java -cp out GUI.ElevatorGUI

```

Listing 2: Kompilacja i uruchomienie projektu.

To sprawi, że w katalogu `src/main/java/out` pojawią się skompilowane klasy projektu. Ostatnia komenda włącza projekt.

4.4 Uruchomienie z IDE

Aby uruchomić program w wybranym IDE, należy wykonać następujące kroki:

1. Otwórz projekt w wybranym środowisku (VS Code, IntelliJ IDEA, Eclipse, NetBeans).
2. Ustaw katalog `src/main/java` jako *source root*.
3. Uruchom metodę `main` w klasie `GUI.ElevatorGUI`.

4.5 Konfiguracja parametrów w GUI

Po uruchomieniu aplikacji powinien wyświetlić się graficzny interfejs użytkownika [sekcji 3.5](#). Jak opisano, w górnej części ekranu znajduje się panel kontrolny - *Parametry symulacji* pozwalający na zmianę podstawowych parametrów windy.

Domyślne parametry po włączeniu programu prezentują się następująco:

Parametr	Domyślna wartość	Opis
Najniższe piętro	-1	Minimalna kondygnacja pionu
Najwyższe piętro	7	Maksymalna kondygnacja pionu
Piętro startowe	4	Pozycja windy na starcie
Pojemność windy	5	Maksymalna liczba pasażerów w kabinie
Czas przejazdu (ms)	1000	Czas podróży między sąsiednimi piętrami
Czas postoju (ms)	3000	Czas oczekiwania przy wsiadaniu/wysiadaniu

Kliknięcie przycisku *Zastosuj i restartuj* zatrzymuje bieżącą symulację, reinicjalizuje model z nowymi parametrami i dodaje domyślnych pasażerów startowych.

5 Architektura i przepływ danych

Projekt stosuje uproszczoną architekturę warstwową:

1. **Model** (`elevator.model`) - cała logika symulacji, niezależna od interfejsu.
2. **Widok/Kontroler** (GUI) - interfejs Swing obsługujący zdarzenia użytkownika i odświeżający widok przez `ElevatorListener`.

Przepływ typowego zdarzenia *wywołanie windy*:

1. Użytkownik klika *Dodaj* lub *Dodaj losowego pasażera* w `ElevatorGUI`.
2. `ElevatorGUI` wywołuje `Division.callElevator(from, to)`.
3. `Division` tworzy `Entity` na odpowiednim `Floor` i dodaje piętro do kolejki `Elevator`.
4. Wątek symulacji (uruchomiony przez `startSimulationThread`) przetwarza kolejkę przez `Elevator.processQueue`.
5. Po każdym ruchu i postoju `Elevator` wywołuje `notifyListeners()`, co uruchamia `ElevatorGUI.updateUIState()` na wątku EDT.

6 Napotkane problemy i błędy

W niniejszej sekcji opisane zostaną problemy i błędy napotkane podczas tworzenia aplikacji symulacyjnej, w kolejności dowolnej.

6.1 Wstępna obsługa kolejki priorytetowej

Wstępnym założeniem projektowym było utworzenie kolejki priorytetowej, która:

- Ma co najwyżej jedną kopię danego piętra w kolejce;
- Wstawia wpiętra wezwane przez pasażerów "wewnątrz windy", potem dojeżdża do pięter które zostały wezwane "z piętra".

Utworzyło to zasadniczy problem - winda jechała wielokrotnie w górę i w dół bez zatrzymania się na odpowiednich piętrach - obsługiwała jednego pasażera na raz.

W takim przypadku istnieje sytuacja, że winda przejedzie wiele iteracji bez zatrzymania się na piętrze gdzie jest osoba oczekujący od dłuższego czasu.

Problem ten jest prostym problemem odpowiedniego ustawienia priorytetów. We wstępnej implementacji wykorzystano najprostszą kolejkę `LinkedList<Integer>`, jednak to nie utrzymuje wzmianki o priorytecie pięter.

W celu rozwiązania problemu zmieniono strukturę kolejki `destinationQueue` na `LinkedList<ArrayList<Integer>>` - to pozwoliło na rozszerzenie wymiaru danych o dodatkowy, w który został wstawiony priorytet kolejki. Wstępnie, priorytet ten miał wartości wyłącznie -1 oraz +1 (gdzie -1 to większy priorytet), ale z uwagi na dodatkowy problem opisany poniżej wymiar ten został rozszerzony.

Ostateczne wartości priorytetu zostały opisane w sekcji [3.3.3](#)

6.2 Omijanie pięter, na których winda mogła się zatrzymać

Problem ten wystąpił po wprowadzeniu priorytetu do kolejki. Objawiał się w sytuacjach analogicznych do podanego poniżej przykładu:

1. Winda zaczyna podróż w dół z 6 piętra do 1 piętra z jednym pasażerem;
2. Na 4 piętrze oczekuje jedna osoba, która jedzie na 2 piętro.

W takiej sytuacji, winda ignorowała osobę na 4 piętrze, i z 6 piętra jechała wprost na 1 piętro aby dokończyć wstępną trasę. Dopiero wtedy, jeżeli żadna osoba nie wsiadła na 1 piętrze, winda jechała do następnej oczekującej w pionie osoby. Analogiczna sytuacja odbywała się, gdy w windzie były 2 lub więcej osoby jadące na różne piętra.

Naprawa błędu wymagała zaktualizowania metody `handleFloorStop` w klasie `Elevator`, oraz wywołania owej metody w `processQueue`.

W przypadku wywołania metody, w `processQueue` nie zwracano uwagi na to, czy aktualne piętro jest *gdziekolwiek* w kolejce, tylko czy jest *na początku*. Poprawiono ten błąd tak, aby winda była zatrzymywana się na piętrze które jest w kolejce priorytetowej, niezależnie od jej kolejności. Aby program działał należało również zaktualizować implementację `handleFloorStop` - rozszerzono o dodatkowe sprawdzanie, czy pasażerowie są na piętrze i odpowiednią obsługę takiego zdarzenia.

Dzięki temu, zmniejszono błędne, wielokrotne przejazdy windy i zbliżono się do możliwej maksymalizacji efektywności przejazdów.

6.3 Długi czas oczekiwania w specyficznych przypadkach

Mimo wcześniej opisanych rozwiązań, pojawił się ostatni napotkany problem z kolejką priorytetową. Miał on do czynienia z faktem, iż wchodząc do windy osoba wstawiała swoje piętro na początek kolejki. Biorąc pod uwagę binarne ograniczenie priorytetu (od -1 do +1) nie dało się zdeterminować tego, która wartość powinna być na początku czy na końcu wyższego priorytetu.

Wówczas, przy wchodzeniu osobnika do windy, priorytet jego piętra został dodany na początek, a cała reszta zignorowana - nawet jeśli oryginalnie winda jechała na niższe piętro. Biorąc przykład z poprzedniego podrozdziału:

1. Winda zaczyna podróż w dół z 6 piętra do 1 piętra z jednym pasażerem;
2. Na 4 piętrze oczekuje jedna osoba, która jedzie na 7 piętro.

w tej sytuacji, winda zatrzymałaby się na piętrze 4 *i pojechała na 7 piętro zamiast zjechać na 1 piętro, gdzie było wcześniej wezwane*.

Tworzyło to kolejny wektor problemu omijania pięter, do którego nie znaleziono rozwiązania z binarnym podziałem priorytetu - więc, aby rozwiązać problem, wprowadzono priorytet -2 - jest to unikalna wartość priorytetu, która zostaje na głowie

kolejki. Daje to możliwość wyznaczenia piętra na które winda *musi* dojechać zanim zmieni kierunek jazdy - rozwiązując wyżej wymieniony problem.

Gdy kolejka dojeżdża do piętra o priorytecie -2, w następnym kroku wyznaczana jest nowe piętro o priorytecie -2 jako głowa kolejki po usunięciu starego piętra.

7 Dalsze możliwości rozwoju

Poniżej przedstawione są proponowane dalsze aspekty rozwoju aplikacji, na które autorzy uważają że należy spojrzeć w pierwszej kolejności:

1. **Tryb automatycznego generowania osobników** - Aktualnie jedyny sposób na dodanie osobników w celu kontynuowania działania windy to ręczne wciśnięcie jednego z dwóch przycisków - dodania z parametrami ustawionymi oraz dodawania z parametrami losowymi. W celu rozszerzenia programu sugeruje się dodanie trybu automatycznego generowania osobników na piętrach, które wykorzysta funkcjonalną pętlę i sprawdzanie liczby istniejących pasażerów do częściowo-realistycznego symulowania codziennego natłoku wykorzystania systemu;
2. **Rozwinięcie wizualizacji jazdy windy** - Z uwagi na możliwość dynamicznego ustawienia pięter w kondygnacji, podczas tworzenia graficznego interfejsu użytkownika skupiono się na efektywnej reprezentacji zmiennej liczby pięter. W celu rozszerzenia programu sugeruje się wprowadzenie dynamicznego pola graficznego, pokazującego windę jako zdjęcie / ikonę poruszającą się na wyznaczonym polu ekranu, rozwinięcie przedstawienia osób oczekujących na piętrze o ikony osób;
3. **Wprowadzenie obsługi wielu wind** - Aplikacja zakłada, że jedna kondygnacja klasy *Division* posiada tylko jedną windę, która obsługuje wszystkich pasażerów - analogicznie do większości klatek bloków mieszkaniowych w Polsce. W celu rozszerzenia programu sugeruje się wprowadzenie obsługi wielu wind, w co wchodzi wspólna kolejka przywołań wind operowana przez wiele obiektów klasy *Elevator*. Wymagałoby to m.in. odpowiedniego oznaczenia pięter, do którego jedzie konkretna winda, aby uniknąć powtarzania niepotrzebnych zatrzymań.

8 Podsumowanie

Aplikacja *Elevator* stworzona w ramach zaliczenia przedmiotu Systemy Operacyjne Czasu Rzeczywistego w odpowiedni sposób symuluje działanie systemu windy obsługującej klatkę / pion pięter w budynku. Dzięki odpowiedniej implementacji oraz z rozszerzeniami obsługującymi faktyczny system automatyczny, aplikacja *Elevator* jest w stanie odpowiednio obsłużyć działanie windy przewozowej, zarówno pasażerskiej jak i towarowej.

W ramach wykonania projektu autorzy byli w stanie bliżej przyjrzeć się procesowi tworzenia systemów operacyjnych czasu rzeczywistego i częstymi problemami z tym procesem związanymi. Było to zarówno istotne w procesie uczenia, jak i ciekawe doświadczenie, które odkryło kolejny aspekt świata cyfryzacji i automatyzacji.

Najważniejszy aspekt całego projektu, tj. stworzenie odpowiedniej kolejki priorytetowej, został wykonany pomyślnie mimo opisanych wcześniej, tymczasowych mankamentów. Dokładając do tego odpowiednie klasy reprezentujące aspekty kwintesencji tematu stworzono system, którego implementację, mimo pozostawionych możliwości dalszego rozwinięcia, można uznać za sukces zespołu, który nad nim pracował.